# Esercizi — cicli: while

**Lezione coperta:** `L06` — il ciclo `while`, la trace table, contatori e accumulatori, un `if` dentro un ciclo, cicli annidati

Variabili, operatori e selezione hanno le loro schede, `L04_esercizi.md` e `L05_esercizi.md`. Qui si dà per saputo tutto quello: il nuovo è **ripetere**.

**Come si lavora.** Tre gradini: prima **seguire** un ciclo e dire che cosa stampa, poi **trovare** l'errore, infine **scriverne** uno. Le soluzioni sono sotto ogni esercizio: aprile dopo aver provato.

**Il primo gradino si fa con carta e penna, con una trace table**: una colonna per variabile, una riga ogni volta che una cambia. È lo strumento dei lucidi `L06_a`, sezione *Trace table*, e serve più nei cicli che in qualsiasi altro punto del corso.

**Gli esercizi del terzo gradino sono quelli dei video `L06_b`**, dove li vedi risolvere passo per passo. Provali prima da solo: il video è per quando sei bloccato, o per confrontare la tua soluzione con un'altra.

---

## Gradino 1 — seguire il codice

## Esercizio 1 — Quante volte, e che cosa (difficoltà: •)

Che cosa stampa? Quante righe?

```
i = 0
while i < 10:
    i = i + 3
    print(i)
```

### Soluzione

Mostra la soluzione

Stampa **quattro** righe: `3`, `6`, `9`, `12`.

| `i` all'inizio del giro | `i < 10`? | `i` dopo `i = i + 3` | stampa |
|---|---|---|---|
| 0 | vero | 3 | 3 |
| 3 | vero | 6 | 6 |
| 6 | vero | 9 | 9 |
| 9 | vero | 12 | 12 |
| 12 | **falso** | — | — |

**Il caso interessante è l'ultimo `12`, che supera il limite.** La condizione si controlla **all'inizio** del giro: quando `i` vale 9 è ancora vera, e dentro il giro `i` diventa 12 e viene stampato prima che qualcuno lo controlli di nuovo. Il `while` non sorveglia la variabile in continuazione: la guarda una volta per giro, sulla soglia.

---

## Esercizio 2 — La trace table (difficoltà: •)

Costruisci la trace table e di' che cosa stampa.

```
n = 5
f = 1
while n > 1:
    f = f * n
    n = n - 1
print(f)
```

### Soluzione

Mostra la soluzione

| passo | `n` | `f` |
|---|---|---|
| inizio | 5 | 1 |
| giro 1 | 4 | 5 |
| giro 2 | 3 | 20 |
| giro 3 | 2 | 60 |
| giro 4 | 1 | 120 |

Con `n = 1` la condizione `n > 1` è falsa: si esce e si stampa **120**.

È il **fattoriale** di 5, `5 · 4 · 3 · 2 = 120`. `f` è un accumulatore che moltiplica invece di sommare, e per questo parte da **1**: partendo da 0 il prodotto resterebbe 0 per sempre.

---

## Esercizio 3 — Quante cifre (difficoltà: •)

Che cosa stampa? E che cosa calcola, in generale, per un intero positivo `n`?

```
n = 4071
cifre = 0
while n > 0:
    n = n // 10
    cifre = cifre + 1
print(cifre)
```

**Soluzione**

Mostra la soluzione

| n | cifre |
|---|---|
| 4071 | 0 |
| 407 | 1 |
| 40 | 2 |
| 4 | 3 |
| 0 | 4 |

Stampa **4**: è il **numero di cifre** di `n`.

`n // 10` toglie l'ultima cifra — la divisione intera di `L04` — e `cifre` conta quante volte si riesce a farlo prima di arrivare a zero. Lo zero in mezzo a 4071 non cambia niente: si contano le cifre, non il loro valore.

Una domanda per chi vuole andare oltre: che cosa stampa con `n = 0`? Zero, perché il ciclo non gira nemmeno — ma lo zero ha una cifra. Per quel caso servirebbe un `if` prima.

---

## Esercizio 4 — Quanti giri (difficoltà: •)

Che cosa stampa? E quante volte si esegue la riga `j = 0`?

```
conta = 0
i = 0
while i < 3:
    j = 0
    while j < 4:
        conta = conta + 1
        j = j + 1
    i = i + 1
print(conta, i, j)
```

**Soluzione**

Mostra la soluzione

Stampa **`12 3 4`**.

Il ciclo **esterno** fa 3 giri, con `i` che vale 0, 1, 2. Per **ciascuno**, quello **interno** ne fa 4. La riga `conta = conta + 1` si esegue quindi `3 · 4 = 12` volte: **i giri si moltiplicano**.

`j = 0` si esegue **3 volte**, una per giro esterno, ed è quello che fa ripartire il ciclo interno ogni volta. Alla fine `i` vale 3 e `j` vale 4: sono i valori che hanno reso **falsa** la condizione, l'ultima volta che è stata controllata.

Per contare i giri di due cicli annidati non serve seguirli tutti: si moltiplicano i giri dell'uno per quelli dell'altro.

---

## Gradino 2 — trovare l'errore

## Esercizio 5 — Non finisce mai (difficoltà: ••)

Il programma dovrebbe fare il conto alla rovescia da 10 a 1. Che cosa fa invece, e perché?

```
i = 10
while i > 0:
    print(i)
    i = i + 1
```

### Soluzione

Mostra la soluzione

Stampa 10, 11, 12, … e **non si ferma più**. Si interrompe con `ctrl + c`.

`i` parte da 10 e **cresce**, quindi `i > 0` resta vera per sempre. Il contatore va nella direzione sbagliata: per un conto alla rovescia serve `i = i - 1`.

La regola che l'errore viola: **dentro il ciclo, qualcosa deve avvicinare la condizione a diventare falsa**. Prima di lanciare un `while`, chiediti quale riga lo farà finire.

---

## Esercizio 6 — Uno di meno (difficoltà: ••)

Il programma dovrebbe stampare i numeri **da 1 a 10**. Funziona?

```
i = 1
while i < 10:
    print(i)
    i = i + 1
```

### Soluzione

Mostra la soluzione

No: stampa da 1 a **9**. Quando `i` vale 10, `i < 10` è falsa e il 10 non viene stampato.

Le correzioni possibili sono due, ed entrambe giuste: `while i <= 10`, oppure `while i < 11`. La prima si legge meglio, perché dice il numero che vuoi raggiungere.

È l'errore più comune nei cicli, e ha un nome: **errore di uno**. Si scopre guardando i **due estremi** — il primo valore stampato e l'ultimo — invece del caso medio. Nei video `L06_a_p6` e `p7` c'è lo stesso errore, trovato con la trace table.

---

## Esercizio 7 — La media sbagliata (difficoltà: ••)

Il programma legge tre numeri e ne stampa la media. Con `4`, `6`, `8` dovrebbe dare 6. Che cosa dà, e dov'è l'errore?

```
n = 0
while n < 3:
    somma = 0
    valore = float(input("numero: "))
    somma = somma + valore
    n = n + 1
print(somma / 3)
```

**Soluzione**

Mostra la soluzione

Stampa **2.6666666666666665**, cioè `8 / 3`.

`somma = 0` sta **dentro** il ciclo: a ogni giro l'accumulatore viene azzerato, e alla fine contiene solo l'ultimo numero letto. Va spostato **prima** del `while`:

```
somma = 0
n = 0
while n < 3:
    valore = float(input("numero: "))
    somma = somma + valore
    n = n + 1
print(somma / 3)
```

Nessun errore, nessun avviso: il programma dà un numero, ed è sbagliato. Un accumulatore si **inizializza una volta**, fuori; dentro, si **aggiorna**.

---

## Esercizio 8 — Cinque numeri, o sette? (difficoltà: ••)

Il programma dovrebbe leggere **cinque** numeri e sommare solo quelli positivi. Con `3`, `-2`, `5`, `-1`, `4` dovrebbe stampare 12. Che cosa fa invece?

```
letti = 0
somma = 0
```

```
while letti < 5:
    x = int(input("numero: "))
    if x > 0:
        somma = somma + x
        letti = letti + 1
print(somma)
```

**Soluzione**

Mostra la soluzione

Dopo il quinto numero **ne chiede ancora**. Con quei cinque valori ha contato solo i tre positivi, quindi `letti` vale 3 e il ciclo continua: chiede numeri finché non ne ha ricevuti **cinque positivi**. Se poi si scrivono `6` e `2`, stampa 20.

`letti = letti + 1` è indentato **dentro l'`if`**, e conta i positivi invece dei numeri letti. La correzione è togliere quattro spazi:

```
letti = 0
somma = 0
while letti < 5:
    x = int(input("numero: "))
    if x > 0:
        somma = somma + x
    letti = letti + 1
print(somma)
```

Ora stampa **12** dopo cinque numeri esatti.

Nessun errore, nessun messaggio: il programma fa una cosa diversa da quella richiesta. Due variabili, due ruoli — `letti` avanza **a ogni numero**, `somma` solo quando l'`if` scatta — e l'indentazione decide quale ruolo ha ciascuna.

---

## Gradino 3 — scrivere

Gli esercizi di questo gradino sono quelli dei video `L06_b`: il numero del video è nel titolo.

## Esercizio 9 — Somma senza Gauss (difficoltà: •••) — video b_p3

Scrivi un programma che chieda un intero `N` e calcoli la somma dei numeri da 1 a `N` **con un ciclo**. Poi verifica il risultato con la formula di Gauss, `N * (N + 1) // 2`.

**Soluzione**

Mostra la soluzione

```
N = int(input("fino a quale numero? "))

somma = 0
i = 1
while i <= N:
    somma = somma + i
    i = i + 1

print("con il ciclo:", somma)
print("con Gauss:   ", N * (N + 1) // 2)
```

Con `N = 100` stampa 5050 tutte e due le volte.

`somma` è l'accumulatore, `i` il contatore. Attenzione a `i <= N`: con `i < N` mancherebbe l'ultimo numero, e i due risultati non coinciderebbero. È l'errore dell'esercizio 6, e la formula di Gauss serve proprio a scoprirlo.

---

## Esercizio 10 — Multipli di 7 o di 9 (difficoltà: •••) — video b_p1

Scrivi un programma che chieda un limite superiore e stampi tutti i numeri, da 2 al limite, che sono multipli di 7 **oppure** di 9.

### Soluzione

Mostra la soluzione

```
limite = int(input("limite superiore: "))

i = 2
while i <= limite:
    if i % 7 == 0 or i % 9 == 0:
        print(i)
    i = i + 1
```

Con `50` stampa 7, 9, 14, 18, 21, 27, 28, 35, 36, 42, 45, 49.

Un `if` dentro il ciclo, e una condizione composta con `or`: 63 è multiplo di tutti e due, e viene stampato **una volta sola**.

Nel video la condizione è `i < limite`, e il limite resta escluso. Non è un errore, è una scelta: «da 2 al limite» si può leggere in due modi. Quello che conta è decidere e **dichiararlo**, per esempio nel messaggio di `input`.

---

## Esercizio 11 — Divisori comuni (difficoltà: •••) — video b_p2

Scrivi un programma che chieda due interi positivi e stampi tutti i loro divisori comuni.

**Soluzione**

Mostra la soluzione

```
a = int(input("primo numero: "))
b = int(input("secondo numero: "))

d = 1
while d <= a and d <= b:
    if a % d == 0 and b % d == 0:
        print(d)
    d = d + 1
```

Con `12` e `18` stampa 1, 2, 3, 6.

Un divisore comune non può superare **il più piccolo** dei due numeri: per questo il ciclo si ferma quando `d` supera l'uno o l'altro. L'ultimo divisore stampato è il **massimo comun divisore**.

---

## Esercizio 12 — Il triangolo di Floyd (difficoltà: •••) — video b_p5

Scrivi un programma che chieda un numero di righe e stampi il triangolo di Floyd: i numeri interi in ordine, uno nella prima riga, due nella seconda, tre nella terza.

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

**Soluzione**

Mostra la soluzione

```
righe = int(input("quante righe? "))

numero = 1
r = 1
while r <= righe:
    riga = ""
    c = 1
    while c <= r:
        riga = riga + f"{numero} "
        numero = numero + 1
        c = c + 1
    print(riga)
    r = r + 1
```

Due cicli annidati: `riga = ""` e `c = 1` stanno dentro quello esterno, perché ogni riga riparte vuota e conta da 1. La differenza è `numero`: è inizializzato **fuori da entrambi** i cicli, e quindi continua a crescere da una riga all'altra invece di ripartire da 1.

Tre variabili, tre ruoli: `r` conta le righe, `c` i numeri dentro la riga, `numero` è il valore da stampare. Riconoscere chi riparte a ogni riga e chi no è tutto l'esercizio.

---

## E il video b_p4, Casuale?

È l'esercizio più lungo, e usa una cosa che non abbiamo ancora visto: numeri estratti a caso, con `import random`. I moduli arrivano in `L08`. Guardalo per il ragionamento — contatori che crescono a seconda del numero uscito — e rifallo quando avremo visto `import`.